

PROOF OF CONCEPT

Waqeel AI

An Enterprise RAG-Based AI Legal Research & Case Preparation Assistant for Law Firms & In-House Legal Teams

Document Code: POC-LAW-005

Classification: Internal — AI Mentor Review

Prepared by: AI Solutions Architecture Team

Version 1.0 — POC Approval Draft

Table of Contents

1. Executive Summary
2. Problem Statement
3. Proposed AI Solution
4. System Architecture
5. Complete Folder Structure
6. Functional Modules
7. APIs
8. Database Design
9. RAG Pipeline
10. LangChain + LangGraph Flow
11. Tech Stack
12. Production Features
13. Future Enhancements
14. Risks
15. Development Roadmap
16. Conclusion

1. Executive Summary

1.1 The Business Problem

Lawyers and in-house legal teams spend a large share of billable and non-billable time manually searching statutes, case law, prior judgments, contracts, and internal precedent files to build arguments, draft opinions, and prepare for hearings. This research is slow, varies in quality with the researching associate's experience, and is difficult to standardize across a firm or legal department.

1.2 Why This Problem Persists

Case law and statutes are updated continuously through new judgments, amendments, and regulatory notifications, spread across court websites, law reporters, and internal case-management systems in inconsistent formats (scanned judgment PDFs, DOCX opinions, Excel precedent trackers). No unified, continuously updated, searchable source of truth exists, so each lawyer or associate re-researches similar questions independently.

1.3 Why Current Solutions Fall Short

Legacy legal-database keyword search returns long lists of loosely relevant judgments rather than a synthesized answer, forcing associates to read dozens of pages to extract the applicable ratio decidendi. Generic chatbots without access to the firm's actual case files and current case law either hallucinate plausible-sounding but non-existent citations or refuse to answer, both of which are professionally dangerous in legal practice.

1.4 Why AI + Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation grounds every answer in the firm's actual statutes, judgments, and case files rather than a model's static training data, which is essential given how frequently case law develops and how damaging a fabricated or outdated citation can be to a legal argument — RAG lets Waqeel AI cite the exact judgment, paragraph, and citation behind every answer.

1.5 Business Value

- Reduces average legal research time from hours to minutes for routine questions.
- Standardizes research quality across associates by grounding every answer in the same authoritative corpus.
- Creates a defensible, reviewable trail of every research query, source, and answer used in a matter.
- Reduces dependency on a small number of senior partners for institutional case knowledge.
- Lowers risk of citing outdated, overruled, or non-existent case law in filings.

1.6 Target Users

Persona	Role	Primary Need
Associate Lawyer	Conducts research and drafts arguments	Fast, cited answers grounded in real case law
Partner	Reviews and approves legal strategy	Traceable citations and confidence scoring
In-House Counsel	Manages contracts and regulatory risk	Quick answers on precedent and internal clause history

Persona	Role	Primary Need
Paralegal	Compiles case files and precedent bundles	Efficient search across firm's document archive

1.7 Expected Impact

- Target: 50-70% reduction in time spent on first-pass legal research.
- Target: 100% of AI-assisted answers traceable to a specific judgment, statute, or contract clause.
- Reduced risk of citing overruled or non-existent case law in filings.
- Faster onboarding of junior associates through self-service Q&A against the firm's precedent library.

2. Problem Statement

2.1 Detailed Problem Description

Law firms and legal departments accumulate large libraries of statutes, judgments, contracts, and internal opinions over years of practice, but this knowledge is fragmented across court databases, shared drives, and the personal case files of individual lawyers, with no unified, queryable system of record connecting a current legal question to the firm's own prior work product.

2.2 Industry Examples

Organization Type	Scenario	Consequence
Mid-sized litigation firm	Associate cites a judgment later discovered to have been overruled, because the internal precedent file was never updated.	Argument weakened in court and credibility questioned by the bench.
Corporate legal department	Junior counsel cannot locate the firm's prior position on a recurring contract clause and re-drafts it inconsistently.	Conflicting contractual language across similar agreements creates future dispute risk.
Boutique advisory firm	Filing deadline research takes two full days because relevant judgments are scattered across three different case-management folders.	Delayed client advice and increased fee burden passed to the client.

2.3 Current Workflow (As-Is)

1. Lawyer receives a legal question tied to an active matter or hearing.
2. Lawyer manually searches legal databases, internal case files, and personal notes.
3. Lawyer cross-references multiple judgments to determine the currently applicable precedent.
4. Lawyer drafts a research memo or argument and routes it to a senior partner for review.
5. Partner verifies the position, often relying on personal recollection of prior matters.
6. Position is finalized and used in filing or advice, with no structured record of sources consulted.

2.4 Pain Points

- No single source of truth for currently good law versus overruled or superseded judgments.
- Heavy reliance on institutional memory held by a small number of senior lawyers.
- Time-consuming manual cross-referencing across statutes, case law, and internal precedent.
- Inconsistent research quality and argument framing across different associates.
- No systematic trail linking a legal position back to the specific authorities relied upon.

2.5 Compliance, Regulatory & Documentation Challenges

Legal positions and filings must be defensible under professional conduct rules, meaning every citation should reflect currently good law and every material argument should be traceable to a specific authority. Courts and bar associations expect diligence in verifying that cited case law has not been overruled, and a firm that cannot demonstrate the basis and currency of its research faces both professional and client-relationship risk.

3. Proposed AI Solution

3.1 Solution Overview

Waqeel AI ingests the firm's or legal department's full corpus of statutes, case law, judgments, contracts, and internal opinions, keeps that corpus continuously updated, and answers lawyer questions in natural language with citations back to the exact judgment, paragraph, or clause relied upon.

3.2 Why Retrieval-Augmented Generation Is the Correct Approach

Because case law develops continuously and the correctness of a legal answer depends entirely on citing currently good law, Waqeel AI must retrieve from a live, versioned corpus rather than rely on a model's frozen training knowledge — RAG is the only architecture that supports this update cadence with source-level traceability essential to legal practice.

3.3 Why a Standard Chatbot Is Not Sufficient

A standard chatbot without retrieval either answers from stale general training data or fabricates plausible-sounding case citations — a well-documented failure mode of ungrounded legal chatbots — which is professionally and ethically unacceptable when the answer informs an actual filing or client advice.

3.4 Why Vector Search Is Required

Legal questions are frequently phrased in plain business or factual language ('can we terminate this vendor contract early') rather than exact legal terminology used in judgments; vector search bridges this gap by matching on semantic meaning, while hybrid keyword search preserves precision for exact citation and section-number lookups.

3.5 How the LLM Improves the Workflow

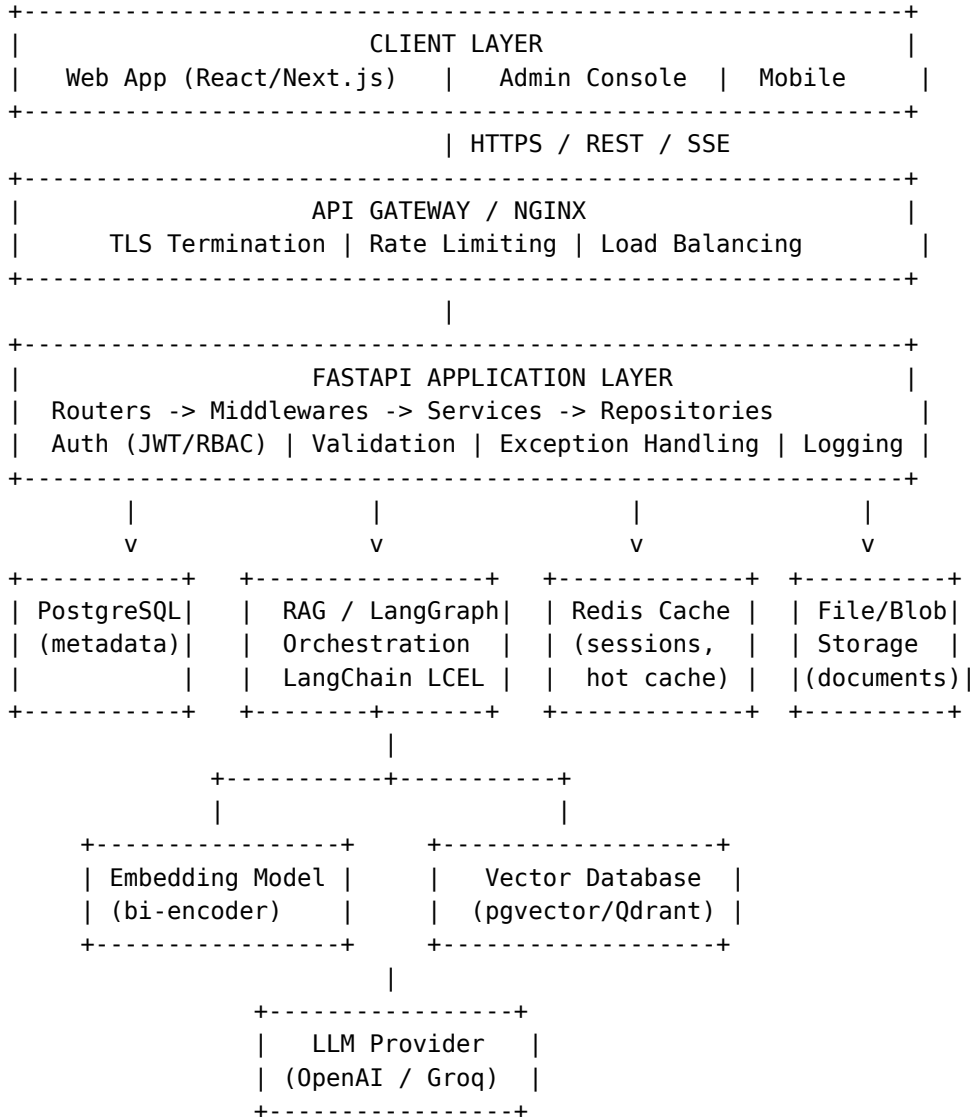
The LLM synthesizes retrieved judgments and clauses into a direct, structured legal research memo, flags conflicting or superseded authority where it exists, and structures the response with explicit citations, turning a multi-document research task into a single conversational query.

4. System Architecture

4.1 Architectural Overview

The Waqeel AI platform is built as a modular, cloud-deployable, layered architecture. Each layer has a single responsibility, communicates through well-defined contracts, and can be scaled or replaced independently. This separation of concerns is essential for an enterprise system that must evolve as legal research and case preparation requirements, models, and regulations change over time.

4.2 High-Level Component Diagram



4.3 Component Responsibilities

Component	Responsibility
Frontend	Chat UI, document upload, admin dashboards, citation rendering, streaming response display.
API Gateway / Nginx	TLS termination, reverse proxy, load balancing, request throttling.

Component	Responsibility
FastAPI Backend	Async REST API layer; routes requests to services; enforces auth and validation.
Authentication (JWT)	Issues and verifies signed access/refresh tokens; short-lived access, rotating refresh.
Role-Based Access Control	Maps users to roles (Admin, Reviewer, Analyst, Viewer) and enforces per-endpoint scopes.
LangChain (LCEL)	Declarative composition of prompt -> retriever -> LLM -> parser as runnables.
LangGraph	Stateful orchestration: conditional routing, retries, human-approval nodes, multi-step agents.
Document Parser	Extracts text/tables from PDF, DOCX, TXT, HTML while preserving structure and page numbers.
Chunking Engine	Splits parsed text into semantically coherent chunks with overlap and metadata tags.
Embedding Model	Converts chunks and queries into dense vectors for semantic similarity search.
Vector Database	Stores embeddings and metadata; supports ANN search and hybrid (keyword + vector) queries.
Retriever	Fetches top-k relevant chunks per query, applies metadata filters (date, doc type, role).
Reranker	Cross-encoder re-scoring of retrieved chunks to improve precision before context injection.
Prompt Template	Structured, versioned prompts with system instructions, guardrails, and citation formatting.
Output Parser	Validates and structures LLM output (JSON schema, citation extraction, refusal detection).
LLM	Generates grounded natural-language answers strictly from the retrieved context.
PostgreSQL	System of record for users, roles, documents, conversations, audit logs, analytics.
Redis	Session cache, rate-limit counters, short-term conversation memory, embedding cache.
File / Blob Storage	Durable storage of original source documents and processed artifacts.
Logging & Monitoring	Structured logs, metrics, tracing across every request for observability and audit.
CI/CD	Automated build, test, container image publishing, and deployment pipeline.

4.4 End-to-End Data Flow

The following sequence describes how a single user query flows through the system, from ingestion of source documents to the delivery of a grounded, cited answer:

- Documents are uploaded via the Document Upload API and stored in Blob Storage.
- The Document Parser extracts raw text and structural metadata (page, section, table).

9. The Chunking Engine splits content into overlapping chunks and tags each with source metadata.
10. The Embedding Model converts each chunk into a vector, which is written to the Vector Database alongside metadata.
11. A user submits a natural-language question through the Chat API.
12. The query is embedded and passed to the Retriever, which performs a hybrid (vector + keyword) search filtered by the user's role and permitted document scope.
13. The Reranker re-orders the top candidate chunks by relevance to the exact query.
14. LangChain (LCEL) assembles a Prompt Template containing the system instructions, the reranked context, and conversation memory.
15. LangGraph routes the request: if confidence is low or the query touches a sensitive workflow, it is routed to a human-approval node before finalization.
16. The LLM generates a grounded response with inline citations back to source chunks.
17. The Output Parser validates the response structure and citation integrity before it is returned to the client.
18. The interaction, retrieved sources, and response are persisted to PostgreSQL for audit and analytics; the response is streamed to the client over SSE.

5. Complete Folder Structure

5.1 Repository Layout

```

src/
  app/
    routers/      # FastAPI route definitions grouped by domain (auth, chat, documents,
admin)           # Business logic orchestrating repositories, RAG pipeline, and external
    services/     # Data access layer – encapsulates all PostgreSQL/ORM queries
calls            # DB session management, connection pooling, Alembic migrations
    repositories/ # SQLAlchemy ORM models (User, Document, Conversation, AuditLog, etc.)
    database/     # Pydantic request/response schemas (DTOs) with validation rules
    models/       # Auth, logging, rate-limiting, correlation-id, exception-handling
    schemas/      # Environment-based settings (Pydantic Settings), secrets loading
    middlewares/  # Security utilities (JWT, hashing), constants, dependency injection
middleware      # Generic helpers (pagination, formatting, retry decorators)
    config/       # Versioned prompt templates (system, retrieval, citation,
    core/         # Embedding client wrapper, batching, caching logic
wiring          # Vector DB client, index management, hybrid-search implementation
    utils/        # Parsers for PDF/DOCX/TXT, chunking strategies
    prompts/      # Retriever, reranker, context-builder, citation-formatter
summarization)   # StateGraph definitions, node functions, conditional edges, agent
    embeddings/   # Unit, integration, and end-to-end tests (pytest)
    vectorstore/  # Local log output (structured JSON, rotated)
    documents/    # FastAPI application entrypoint / ASGI app factory
    rag/          # Database migration scripts
    langgraph/    # Dockerfile(s), docker-compose for local/dev stacks
flows           # CI/CD pipeline definitions (GitHub Actions)
    tests/        # React/Next.js client application
    logs/         # Chat window, citation cards, upload widget, admin tables
    main.py       # Route-level pages
alembic/         # Data-fetching and auth hooks
docker/         # Documented environment variable template
.github/workflows/ # Python dependency and tooling configuration
frontend/       # Architecture overview and setup instructions
  src/components/
  src/pages/
  src/hooks/
.env.example
pyproject.toml
README.md

```

5.2 Purpose of Each Folder

Folder	Purpose
routers/	Thin HTTP layer; parses requests, calls services, returns schemas. No business logic.
services/	Coordinates use cases (e.g. 'ask a question', 'ingest a document'); the core business logic.
repositories/	Isolates all SQL/ORM access so services never talk to the database directly.
database/	Engine/session setup, connection pooling, and Alembic migration configuration.
models/	Declarative ORM entities mapped to PostgreSQL tables.

Folder	Purpose
schemas/	Pydantic models defining and validating API request/response contracts.
middlewares/	Cross-cutting concerns applied to every request: auth, logging, rate limits.
config/	Centralized, environment-aware configuration (dev/staging/prod).
core/	Security primitives, dependency injection, shared constants.
utils/	Reusable helper functions not tied to a specific domain.
prompts/	Prompt templates kept out of code, versioned, and independently testable.
embeddings/	Wraps the embedding provider; adds batching, retries, and caching.
vectorstore/	Abstracts the vector database so the provider can be swapped with minimal change.
documents/	Format-specific parsing and chunking logic.
rag/	The retrieval-augmented generation pipeline: retrieve, rerank, build context, cite.
langgraph/	Stateful multi-step orchestration graphs and node implementations.
tests/	Automated test suite covering unit, integration, and RAG-quality evaluation.

6. Functional Modules

The platform is decomposed into the following functional modules. Each module is independently testable and maps to a bounded context within the legal research and case preparation domain.

Authentication

Enterprise SSO-compatible login with JWT-based session management for all legal staff.

User Management

Provisioning of associates, partners, paralegals, and in-house counsel with matter-scoped access.

Admin Dashboard

Corpus health monitoring, ingestion status, and usage oversight for platform administrators.

Document Upload

Ingestion of statutes, judgments, contracts, and internal opinions in PDF, DOCX, and TXT.

Document Processing

Parsing, structure extraction, and good-law/overruled status tagging of every ingested authority.

Embedding Pipeline

Automated chunking and vectorization of new judgments, statutes, and contract templates.

Vector Database

Stores embeddings with jurisdiction, court level, and citation-status metadata for precise filtering.

Chat Module

Conversational interface for natural-language legal questions with streamed, cited answers.

Conversation Memory

Retains prior turns in a research session so follow-up questions stay contextual to the matter.

Search

Direct hybrid search across the legal corpus outside the chat flow, for power users.

Audit Logs

Immutable record of every question, retrieved source, and answer for professional-conduct defensibility.

Notifications

Alerts lawyers when a judgment relevant to a saved matter is overruled or amended.

Analytics

Dashboards on most-researched issue categories, matter types, and unresolved query patterns.

Reports

Exportable research memos and precedent bundles to support filings and client advice.

Settings

Jurisdiction scope, notification preferences, and default matter configuration per user.

Role Management

Fine-grained RBAC distinguishing Associate, Partner, In-House Counsel, and Admin permissions.

7. APIs

7.1 API Design Principles

- RESTful resource-oriented URLs (e.g. /api/v1/documents/{id}).
- Consistent envelope for success and error responses.
- Cursor and offset pagination supported on all list endpoints via ?page & ?page_size.
- Filtering via query parameters (?status=, ?doc_type=, ?date_from=, ?date_to=).
- Sorting via ?sort_by= and ?order=asc|desc.
- Idempotent PUT/DELETE; partial updates via PATCH where applicable.
- All endpoints versioned under /api/v1/ to allow non-breaking evolution.

7.2 Core Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user, issue JWT access + refresh tokens.
POST	/api/v1/auth/refresh	Exchange a valid refresh token for a new access token.
GET	/api/v1/users/me	Return the authenticated user's profile and role.
POST	/api/v1/documents	Upload a new source document for ingestion.
GET	/api/v1/documents	List documents with pagination, filtering, and sorting.
GET	/api/v1/documents/{id}	Retrieve a single document's metadata and processing status.
DELETE	/api/v1/documents/{id}	Soft-delete a document and its vector embeddings.
POST	/api/v1/chat/sessions	Create a new conversation session.
POST	/api/v1/chat/sessions/{id}/messages	Submit a question; returns a streamed, cited answer.
GET	/api/v1/chat/sessions/{id}	Retrieve full conversation history.
GET	/api/v1/audit-logs	List audit log entries (Admin only), paginated and filterable.
GET	/api/v1/analytics/usage	Aggregate usage metrics for dashboards.

7.3 Standard Response Envelope

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "page_size": 20, "total_items": 134, "total_pages": 7 },
  "error": null
}
```

7.4 Standard Error Response

```
{
  "success": false,
  "data": null,
  "meta": null,
  "error": {
    "code": 400,
    "message": "Invalid request"
  }
}
```

```

    "error": {
      "code": "DOCUMENT_NOT_FOUND",
      "message": "No document exists with the given ID.",
      "request_id": "3f9a2c11-..."
    }
  }
}

```

7.5 Sample Request / Response — Ask a Question

Request:

POST /api/v1/chat/sessions/8f21/messages

Authorization: Bearer <jwt>

Content-Type: application/json

```

{
  "message": "Is early termination of a vendor contract for repeated late delivery enforceable under our standard agreement, based on prior firm precedent?"
}

```

Response:

```

{
  "success": true,
  "data": {
    "message_id": "9c31-...",
    "answer": "Yes; the firm's standard vendor agreement template includes a material-breach termination clause that prior precedent has upheld for repeated late delivery, provided a documented cure notice was issued and the cure period lapsed without remedy.",
    "citations": [
      { "source": "Precedent_Memo_Vendor_Termination_2025.docx", "page": 4 }
    ],
    "confidence": "high"
  },
  "error": null
}

```

7.6 HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PUT/PATCH.
201	Created	Successful POST creating a resource.
204	No Content	Successful DELETE.
400	Bad Request	Validation failure on request payload.
401	Unauthorized	Missing or invalid JWT.
403	Forbidden	Authenticated but insufficient role/permission.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate resource or state conflict.
422	Unprocessable Entity	Semantically invalid request (e.g. unsupported file type).
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	Unhandled server-side exception.

8. Database Design

8.1 Core Tables

Table	Key Columns	Notes
users	id (PK), email, hashed_password, role_id (FK), created_at	One row per authenticated user.
roles	id (PK), name, permissions (JSONB)	Defines RBAC permission sets.
documents	id (PK), title, doc_type, storage_uri, status, uploaded_by (FK users), created_at	Metadata for every ingested source document.
document_chunks	id (PK), document_id (FK), chunk_index, content, token_count, page_number	Chunk-level metadata; vectors live in the vector store, keyed by chunk id.
conversations	id (PK), user_id (FK), title, created_at	One row per chat session.
messages	id (PK), conversation_id (FK), role, content, created_at	Individual turns within a conversation.
citations	id (PK), message_id (FK), document_chunk_id (FK), relevance_score	Links an answer to the exact source chunks used.
audit_logs	id (PK), user_id (FK), action, resource_type, resource_id, ip_address, created_at	Immutable record of security-relevant actions.
notifications	id (PK), user_id (FK), type, payload (JSONB), read_at	In-app notification queue.

8.2 Relationships

- users 1—N documents (uploaded_by)
- users 1—N conversations
- roles 1—N users
- documents 1—N document_chunks
- conversations 1—N messages
- messages 1—N citations
- document_chunks 1—N citations

8.3 Indexing Strategy

- B-tree index on users.email (unique) for fast login lookup.
- Composite index on document_chunks(document_id, chunk_index) for ordered retrieval.
- Index on audit_logs(created_at) to support time-bounded compliance queries.
- Index on messages(conversation_id, created_at) for chronological history retrieval.
- GIN index on documents.metadata (JSONB) for flexible attribute filtering.

8.4 Normalization

The schema is normalized to Third Normal Form (3NF) to eliminate redundant storage of user, document, and conversation attributes. Denormalized read-optimized views are introduced only where analytics dashboards

require aggregate performance, and are refreshed on a schedule rather than stored redundantly in transactional tables.

8.5 Entity-Relationship Summary

```
roles ---< users ---< documents ---< document_chunks ---< citations >--- messages ---<
conversations
users ---< audit_logs
users ---< notifications
```

9. RAG Pipeline

9.1 Document Loader

Format-specific loaders normalize heterogeneous inputs into a common internal representation preserving layout metadata (page, section heading, table boundaries):

Format	Loader Strategy
PDF	Layout-aware extraction (PyMuPDF/pdfplumber) preserving page numbers and table structure; OCR fallback for scanned pages.
DOCX	python-docx traversal preserving headings, tables, and styles for section-aware chunking.
TXT	Encoding-safe text read with paragraph-boundary detection.

9.2 Chunking

Recursive, structure-aware chunking targets 400–800 tokens per chunk with a 10–15% overlap, splitting first on section/heading boundaries, then paragraphs, then sentences, so that no chunk crosses a semantic boundary unnecessarily. Each chunk retains metadata: source document, page, section title, and effective date.

9.3 Embeddings

Chunks and queries are embedded using a bi-encoder sentence-transformer (or an API-based embedding model) into a shared vector space. Embeddings are batched and cached to control latency and cost, and are re-generated whenever the underlying model version changes.

9.4 Vector Database

Vectors are stored with associated metadata in a vector database (pgvector for tight PostgreSQL integration, or a dedicated store such as Qdrant for larger scale), enabling approximate nearest-neighbor search with metadata pre-filtering by document type, role scope, and date range.

9.5 Retriever & Hybrid Search

The retriever combines dense vector similarity with sparse keyword (BM25) search, merging results via reciprocal rank fusion. Hybrid search is critical in this domain because exact terminology, identifiers, and section numbers must be matched precisely, while semantic search captures paraphrased user intent.

9.6 Reranker

A cross-encoder reranker re-scores the top-N hybrid candidates against the exact user query, materially improving precision before the most relevant 4–8 chunks are passed into the LLM's context window.

9.7 Context Injection & Prompt Engineering

Reranked chunks are assembled into a structured context block with explicit source labels. The system prompt instructs the model to answer strictly from the provided context, to state when information is not available, and to attach a citation to every factual claim.

9.8 LLM Response & Citation

The LLM generates an answer grounded in the injected context. The Output Parser cross-checks that every citation in the response maps to an actual retrieved chunk, rejecting or flagging any answer with unverifiable references before it reaches the user.

9.9 Source References

Every answer displays the originating document title, page number, and an excerpt snippet, allowing the user to verify the answer against the primary source in one click — a requirement for any regulated or audit-sensitive workflow.

9.10 Conversation Memory

Short-term memory (the current session's turns) is held in Redis and included in the prompt for coherence. Long-term memory (summarized prior sessions) is persisted in PostgreSQL and selectively retrieved when relevant to the current query.

10. LangChain + LangGraph Flow

10.1 LCEL (LangChain Expression Language)

The pipeline is composed declaratively as a chain of Runnable: prompt | retriever | llm | output_parser. LCEL provides a uniform interface (invoke, batch, stream) across every step, making the pipeline composable, testable in isolation, and easy to reason about.

```
chain = prompt_template | retriever | reranker | llm | output_parser

chain.invoke(query)          # synchronous, single request
chain.batch([q1, q2, q3])    # parallel batch processing
await chain.ainvoke(query)    # async, single request (used in FastAPI routes)
await chain.abatch([...])    # async batch (bulk re-indexing / evaluation jobs)
```

10.2 StateGraph (LangGraph)

For workflows that require more than a single retrieve-then-generate pass, LangGraph models the interaction as a state machine. Each node performs one responsibility (classify intent, retrieve, rerank, generate, validate); edges route the state to the next node based on runtime conditions.

```
START -> classify_intent
classify_intent -> [ retrieve_context | direct_answer | escalate ]    (conditional edge)
retrieve_context -> rerank -> generate_answer
generate_answer -> validate_citations
validate_citations -> [ END | human_approval ]                      (conditional edge)
human_approval -> END
```

10.3 Conditional Edges

Conditional edges inspect the current state (confidence score, document sensitivity flag, citation completeness) and route execution accordingly — for example, sending low-confidence or high-stakes answers to a human-approval node instead of returning them directly to the user.

10.4 Agentic Flow

For multi-step questions (e.g. requiring lookups across multiple document types or a calculation step), LangGraph enables an agent loop where the model can call retrieval or calculation tools iteratively before composing a final answer, with a bounded step count to prevent runaway loops.

10.5 Memory

LangGraph's checkpointing persists the full state of a conversation, enabling pause/resume for human-approval steps and reliable recovery if a node fails mid-execution.

10.6 Fallback Strategy

If the primary LLM provider errors or times out, the graph falls back to a secondary provider or a cached response strategy, and surfaces a clear degraded-mode indicator to the user rather than failing silently.

10.7 Human-in-the-Loop Approval

For high-stakes outputs, LangGraph pauses execution at a `human_approval` node, notifying a reviewer through the Notifications module; the graph resumes only after explicit reviewer sign-off, which is itself recorded in the audit log.

11. Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js, TypeScript, Tailwind	Chat interface, document management, admin dashboards.
Backend Framework	FastAPI (Python 3.11+)	Async REST API layer.
Orchestration	LangChain (LCEL) + LangGraph	RAG pipeline composition and stateful workflow control.
Primary LLM	OpenAI GPT-4-class models	Answer generation, summarization, structured extraction.
Secondary LLM	Groq-hosted open models	Low-latency fallback and cost-sensitive workloads.
Embedding Model	Sentence-Transformers / OpenAI text-embedding	Dense vector representation of text.
Vector Database	pgvector (or Qdrant for scale)	Semantic similarity search.
Relational Database	PostgreSQL 15+	System of record for users, documents, conversations, audit.
Cache / Session Store	Redis	Caching, rate limiting, session and short-term memory.
Object Storage	S3-compatible blob storage	Durable storage of source documents.
Reverse Proxy	Nginx	TLS termination, load balancing.
Containerization	Docker	Reproducible builds and deployment artifacts.
CI/CD	GitHub Actions	Automated test, build, and deployment pipeline.
Cloud Deployment	AWS / Azure (Kubernetes or ECS)	Managed, autoscaling production hosting.
Observability	OpenTelemetry, Prometheus, Grafana	Metrics, tracing, dashboards, alerting.

12. Production Features

Feature	Implementation Detail
JWT Authentication	Short-lived signed access tokens (15 min) with rotating refresh tokens (7 days), stored httpOnly.
Role-Based Access Control	Declarative permission checks per endpoint, enforced via FastAPI dependencies.
Logging	Structured JSON logs with correlation IDs, shipped to a centralized log store.
Exception Handling	Global exception handlers mapping domain errors to consistent HTTP responses.
Monitoring	OpenTelemetry traces + Prometheus metrics on latency, error rate, and token usage.
Retry Mechanism	Exponential backoff with jitter on LLM/embedding provider calls.
Rate Limiting	Per-user and per-IP token-bucket limits enforced at the gateway and application layer.
Caching	Redis caching of embeddings, frequent queries, and session data.
Pagination	Cursor/offset pagination on every list endpoint with configurable page size.
Search	Hybrid vector + keyword search with metadata filters.
Async APIs	Fully async FastAPI routes and DB access (asyncpg / SQLAlchemy async).
Streaming Response	Server-Sent Events streaming of LLM tokens for perceived low latency.
File Validation	MIME-type, size, and malware-scan checks on every upload.
Security	Encryption in transit (TLS) and at rest, secrets management, input sanitization.
Audit Logs	Immutable, append-only log of every access and administrative action.
Scalability	Stateless API pods behind a load balancer; horizontal autoscaling on CPU/queue depth.

13. Future Enhancements

- Automated monitoring and ingestion of new judgments as they are published by relevant courts.
- Multi-jurisdiction expansion with jurisdiction-aware routing of research questions.
- Draft-generation assistance for pleadings and contract clauses grounded in firm precedent.
- Integration with the firm's case-management system to link research directly to active matters.
- Fine-tuned domain-specific reranker trained on partner-validated research feedback.

14. Risks

14.1 Technical Risks

Risk	Description	Mitigation
Overruled or superseded authority	A cited judgment is later overruled but remains in the retrievable corpus.	Automated citation-status monitoring with flags on superseded authority.
Fabricated citations	Ungrounded generation could reference a non-existent case, a known risk in legal AI.	Strict retrieval-grounding with citation-integrity validation before any answer is returned.
Matter confidentiality breach	Cross-matter or cross-client data leakage between legal teams.	Metadata-enforced retrieval filtering plus RBAC tested in CI, aligned to ethical-wall requirements.
Vendor/API outage	Primary LLM provider downtime blocks the research workflow during a filing deadline.	Secondary LLM provider fallback with automatic failover.

14.2 AI Hallucination Risk

Because legal positions directly inform court filings, client advice, and professional-conduct obligations, an ungrounded or fabricated answer carries real business and compliance risk. The system mitigates this through strict context-grounding (the model is instructed never to answer outside retrieved context), mandatory citation validation, confidence scoring, and human-approval routing for high-stakes responses.

14.3 Mitigation Summary

- Retrieval-grounded generation with explicit 'answer only from context' instructions.
- Citation-integrity validation before any response reaches the user.
- Confidence thresholds that route uncertain answers to human review.
- Continuous evaluation against a labeled question/answer regression set.
- Versioned prompts and models with rollback capability.

15. Development Roadmap

Phase	Weeks	Deliverables
POC	1-3	Corpus ingestion pipeline for a single jurisdiction; core RAG chat with citations; internal demo.
MVP	4-8	Multi-document-type support, RBAC, audit logging, analytics dashboard, pilot with one practice group.
Production Hardening	9-11	Load testing, security review, monitoring/alerting, CI/CD pipeline, DR plan.
Deployment	12	Staged rollout to the full firm or legal department with training and feedback loop.

16. Conclusion

Waqeel AI directly addresses a high-cost, high-risk manual process with a well-understood, auditable AI architecture. Because legal research is fundamentally a document-grounded question-answering problem, RAG is a strong architectural fit, and the value — faster research, consistent argument quality, and a defensible citation trail — is both measurable and immediate.

Law firms and legal departments with active litigation and advisory caseloads stand to gain measurable reductions in research time and citation risk, making this a strong candidate for continued investment beyond the POC stage into a production-grade, continuously updated legal research platform.

Prepared for approval review — AI Mentor sign-off requested prior to POC development kickoff.